

DEPARTMENT OF COMPUTER SCIENCE

University of Karachi

Course Supervisor: Dr. Humera Tariq

Date: 14 July 2026

OBJECT-ORIENTED PROGRAMMING

WEEK 1 LAB MANUAL

Thinking in Objects: From Everyday Things to Java Classes

Everyday objects become software models

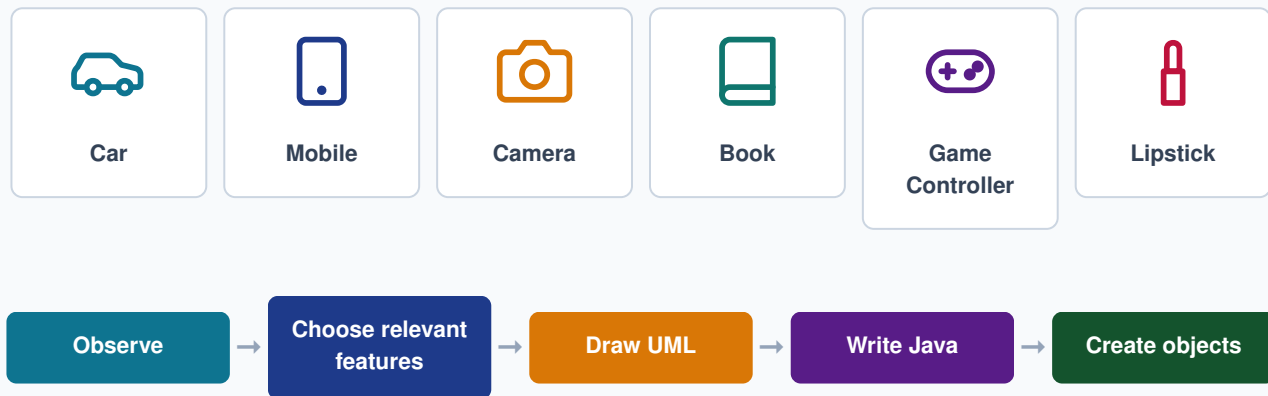


Figure 1. Everyday objects can be described through attributes and behaviours, then represented as Java classes.

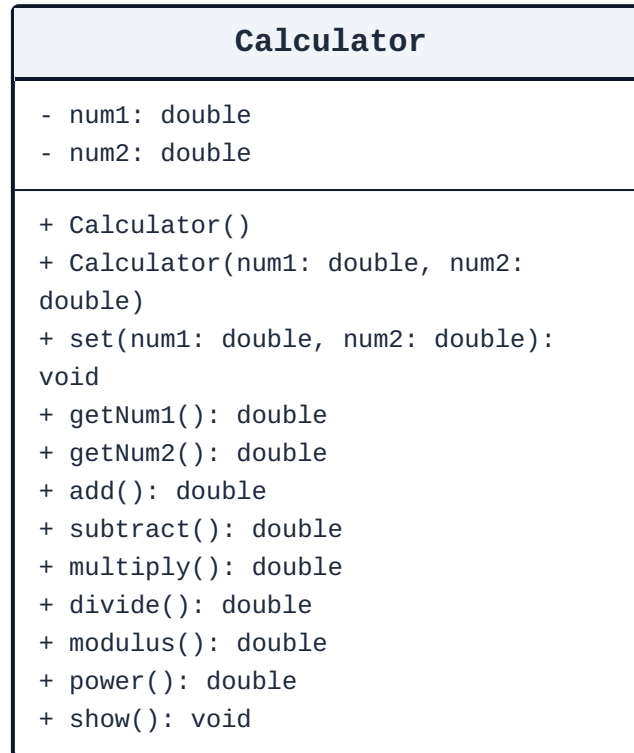
Semester	July–November 2026	Credit Hours	4 (3 Theory + 1 Lab)
Sections	A, B and C	Lab Number	01

Student Name	<u>Aiman Nafees</u>	Seat No.	<u>B25110006012</u>
Section	<u>bscs(b)</u> Selection: ☐ A ☒ B ☐ C	Date	<u>25-july-2026</u>

Student worksheet

#	Favourite object	Relevant attributes (what it has)	Behaviours (what it can do)
1	Calculator	num1: double, num2: double	Calculator(), Calculator(num1, num2), set(num1, num2), getNum1(), getNum2(), add(), subtract(), multiply(), divide(), modulus(), power(), show()
2	Car	brand: String, color: String, speed: int, engineOn: boolean, fuelLevel: double, direction: String, autopilotEnabled: boolean, seatBeltFastened: boolean, wipersOn: boolean, solarChargingEnabled: boolean, fuelEfficiency: double	Car(), setDetails(brand, color), startEngine(), stopEngine(), drive(), moveFast(), moveSlow(), moveLeft(), moveRight(), toggleAutopilot(), checkSeatBelt(fastened), autoWiper(isRaining), voiceCommand(command), calculateRange(), generateSolarPower(), toggleSolarCharging(), displayState()
3	Smart Lock	fingerLock: boolean, faceRecognition: boolean, batteryStatus: String, phoneConnection: boolean, connectivity: String (bluetooth/wifi), surveillanceCamera: boolean	checkFingerprint(), recognizeFace(), checkBatteryHealth(), notifyBatteryReplacement(), connectPhone(), recordVideo()
4	Robotic Vacuum	state: String, batteryLevel: double, binFull: boolean, roomMapping: String	startCleaning(): void, stop(): void, returnToDock(): void, emptyBin(): void, changeMode(mode: String): void
5	Drone	altitudeHeight: double, batteryLevel: double, speed: int, gps: String, camera: boolean, flightMode: String (manual/autopilot)	takeOff(): void, land(): void, distanceCovered(left/right), setRecord(camera: boolean), returnHome(): void, voiceCommand()

1. UML CLASS DIAGRAM



2. SOURCE CODE

Calculator.java

```
public class Calculator {
    private double num1;
    private double num2;

    public Calculator() {
        this.num1 = 0;
        this.num2 = 0;
    }

    public Calculator(double num1, double num2) {
        this.num1 = num1;
        this.num2 = num2;
    }

    public void set(double num1, double num2) {
        this.num1 = num1;
        this.num2 = num2;
    }

    public double getNum1() { return this.num1; }
    public double getNum2() { return this.num2; }

    public double add() { return this.num1 + this.num2; }
    public double subtract() { return this.num1 - this.num2; }
    public double multiply() { return this.num1 * this.num2; }

    public double divide() {
        if (this.num2 == 0) {
            System.out.println("Error: Cannot divide by zero.");
            return 0;
        }
    }
}
```

```

        return this.num1 / this.num2;
    }

    public double modulus() {
        if (this.num2 == 0) {
            System.out.println("Error: Cannot compute modulus with zero.");
            return 0;
        }
        return this.num1 % this.num2;
    }

    public double power() {
        return Math.pow(this.num1, this.num2);
    }

    public void show() {
        System.out.println("num1 = " + this.num1 + ", num2 = " + this.num2);
    }
}

```

CalculatorMain.java

```

public class CalculatorMain {
    public static void main(String[] args) {
        Calculator calc = new Calculator(10, 3);
        calc.show();

        System.out.println("Add: " + calc.add());
        System.out.println("Subtract: " + calc.subtract());
        System.out.println("Multiply: " + calc.multiply());
        System.out.println("Divide: " + calc.divide());
        System.out.println("Modulus: " + calc.modulus());
        System.out.println("Power: " + calc.power());

        Calculator edgeCalc = new Calculator(5, 0);
        edgeCalc.show();
        System.out.println("Power edge case: " + edgeCalc.power());
    }
}

```

3. PROGRAM EXECUTION OUTPUT

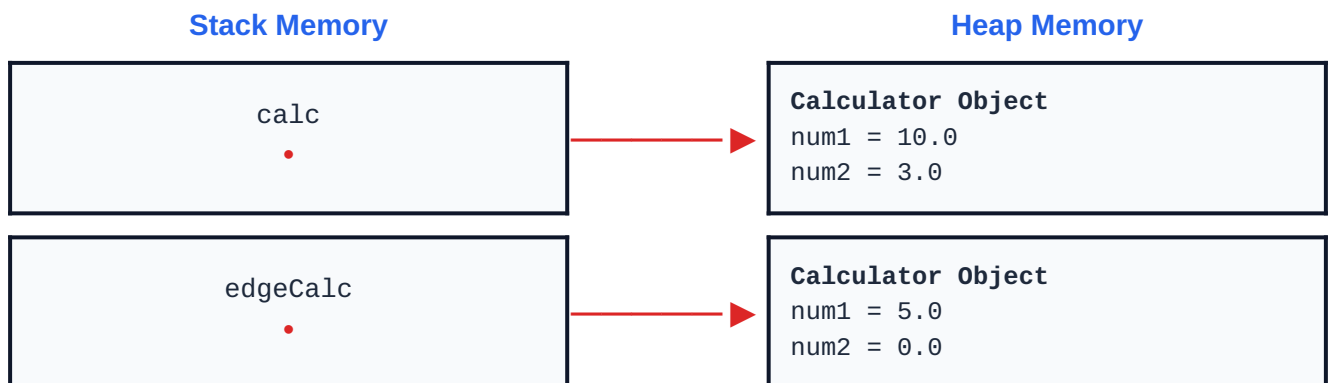
```

num1 = 10.0, num2 = 3.0
Add: 13.0
Subtract 7.0
Multiply: 30.0
Divide: 3.3333333333333335
Modulus: 1.0
Power: 1000.0

num1 = 5.0, num2 = 0.0
Power edge case: 1.0

```

4. MEMORY ALLOCATION (STACK & HEAP)



JAVA OBJECT-ORIENTED PROGRAMMING

Car Class Architecture, Source Code, Execution Output & Memory Model

1. CLASS DIAGRAM



2. SOURCE CODE IMPLEMENTATION

Car.java

```
public class Car {
    private String brand;
    private String color;
    private int speed;
    private boolean engineOn;
    private double fuelLevel;
    private String direction;
    private boolean autopilotEnabled;
    private boolean seatBeltFastened;
    private boolean wipersOn;
    private boolean solarChargingEnabled;
    private double fuelEfficiency; // km travelled per unit of fuel

    public Car() {
        this.brand = "Unknown";
        this.color = "Unknown";
        this.speed = 0;
        this.engineOn = false;
        this.fuelLevel = 100.0;
        this.direction = "straight";
        this.autopilotEnabled = false;
    }
}
```

```

        this.seatBeltFastened = false;
        this.wipersOn = false;
        this.solarChargingEnabled = false;
        this.fuelEfficiency = 15.0;
    }

    public void setDetails(String brand, String color) {
        this.brand = brand;
        this.color = color;
    }

    public void startEngine() {
        if (!this.engineOn) {
            this.engineOn = true;
            System.out.println(this.brand + "'s engine started.");
        } else {
            System.out.println(this.brand + "'s engine is already running.");
        }
    }

    public void stopEngine() {
        this.engineOn = false;
        this.speed = 0;
        System.out.println(this.brand + "'s engine stopped. Speed reset to 0.");
    }

    public void drive() {
        if (!this.engineOn) {
            System.out.println(this.brand + " cannot drive - engine is off.");
            return;
        }
        this.speed += 10;
        this.fuelLevel = Math.max(0, this.fuelLevel - 1);
        System.out.println(this.brand + " is driving at " + this.speed + " km/h.");
    }

    public void moveFast() {
        if (!this.engineOn) {
            System.out.println(this.brand + " cannot accelerate - engine is off.");
            return;
        }
        this.speed += 20;
        System.out.println(this.brand + " speeds up to " + this.speed + " km/h.");
    }

    public void moveSlow() {
        this.speed = Math.max(0, this.speed - 10);
        System.out.println(this.brand + " slows down to " + this.speed + " km/h.");
    }

    public void moveLeft() {
        this.direction = "left";
        System.out.println(this.brand + " is turning left.");
    }

    public void moveRight() {
        this.direction = "right";
        System.out.println(this.brand + " is turning right.");
    }

    public void toggleAutopilot() {
        if (!this.engineOn) {
            System.out.println(this.brand + " cannot enable autopilot - engine is off.");
            return;
        }
        this.autopilotEnabled = !this.autopilotEnabled;
        System.out.println(this.brand + "'s autopilot is now "
            + (this.autopilotEnabled ? "ON" : "OFF") + ".");
    }

    public void checkSeatBelt(boolean fastened) {
        this.seatBeltFastened = fastened;
    }

```

```

        if (!this.seatBeltFastened) {
            System.out.println(this.brand + ": no seat belt detected - siren activated!");
        } else {
            System.out.println(this.brand + ": seat belt fastened.");
        }
    }

    public void autoWiper(isRaining) {
        this.wipersOn = isRaining;
        if (isRaining) {
            System.out.println(this.brand + ": rain detected - wipers turned on automatically.");
        } else {
            System.out.println(this.brand + ": no rain - wipers off.");
        }
    }

    public void voiceCommand(String command) {
        System.out.println(this.brand + " received voice command: " + command + "");
        switch (command.toLowerCase()) {
            case "start engine":
                startEngine();
                break;
            case "stop engine":
                stopEngine();
                break;
            case "drive":
                drive();
                break;
            default:
                System.out.println(this.brand + ": command not recognized.");
        }
    }

    public double calculateRange() {
        double range = this.fuelLevel * this.fuelEfficiency;
        System.out.println(this.brand + " can travel approximately " + range + " km with current fuel.");
        return range;
    }

    public void toggleSolarCharging() {
        this.solarChargingEnabled = !this.solarChargingEnabled;
        System.out.println(this.brand + "'s solar charging is now "
            + (this.solarChargingEnabled ? "enabled" : "disabled") + ".");
    }

    public void generateSolarPower() {
        if (this.solarChargingEnabled) {
            this.fuelLevel = Math.min(100, this.fuelLevel + 5);
            System.out.println(this.brand + "'s solar panels generated extra charge. Fuel level: "
                + this.fuelLevel);
        } else {
            System.out.println(this.brand + "'s solar charging is disabled.");
        }
    }

    public void displayState() {
        System.out.println("---- Car state ----");
        System.out.println("Brand: " + this.brand);
        System.out.println("Color: " + this.color);
        System.out.println("Speed: " + this.speed + " km/h");
        System.out.println("Engine on: " + this.engineOn);
        System.out.println("Fuel level: " + this.fuelLevel);
        System.out.println("Direction: " + this.direction);
        System.out.println("Autopilot: " + this.autopilotEnabled);
        System.out.println("Seat belt fastened: " + this.seatBeltFastened);
        System.out.println("Wipers on: " + this.wipersOn);
        System.out.println("Solar charging enabled: " + this.solarChargingEnabled);
        System.out.println("-----");
    }
}

```


Main.java

```
public class Main {
    public static void main(String[] args) {
        System.out.println("== Car 1 ==");
        Car car1 = new Car();
        car1.setDetails("Toyota", "Red");

        car1.drive();
        car1.startEngine();
        car1.drive();
        car1.moveFast();
        car1.moveLeft();
        car1.toggleAutopilot();
        car1.checkSeatBelt(false);
        car1.autoWiper(true);
        car1.voiceCommand("drive");
        car1.calculateRange();

        car1.displayState();
    }
}
```

3. CONSOLE EXECUTION OUTPUT

```
aiman@aiman-Lenovo-V310-15IKB:~$ /usr/bin/env /usr/lib/jvm/jdk-26.0.1-oracle-x64/bin/java --enable-preview -XX:+ShowCodeDetailsInExceptionMessages -cp /tmp/vscodesws_b4f84/jdt_ws/jdt.ls-java-project/bin Main
```

```
== Car 1 ==
Toyota cannot drive - engine is off.
Toyota's engine started.
Toyota is driving at 10 km/h.
Toyota speeds up to 30 km/h.
Toyota is turning left.
Toyota's autopilot is now ON.
Toyota: no seat belt detected - siren activated!
Toyota: rain detected - wipers turned on automatically.
Toyota received voice command: "drive"
Toyota is driving at 40 km/h.
Toyota can travel approximately 1470.0 km with current fuel.
---- Car state ----
Brand: Toyota
Color: Red
Speed: 40 km/h
Engine on: true
Fuel level: 90.0
Direction: left
Autopilot: true
Seat belt fastened: false
Wipers on: true
Solar charging enabled: false
-----
```

4. STACK AND HEAP MEMORY DIAGRAM

CAR MEMORY ALLOCATION DIAGRAM

Stack and Heap Memory Representation for a Single Car Object

Stack Memory

myCar



Heap Memory

Car Object

```
brand = "Tesla"
color = "Red"
speed = 60
engineOn = true
fuelLevel = 85.5
direction = "North"
autopilotEnabled = false
seatBeltFastened = true
wipersOn = false
solarChargingEnabled = true
fuelEfficiency = 15.0
```